Praxis Trading Platform

UI Walkthrough — Partner Brief (Redesign)

This document walks through the redesigned Praxis dashboard one surface at a time. The frontend was rebuilt over Phases 1–9 of the redesign program; the merged design portfolio (`docs/design/`) governs visual and IA decisions. Every behavior described below is grounded in source code — file paths and line ranges are listed under each section.

The redesign is organized around **five canonical surfaces** and a shared chrome:

- **Hot Trading** — `/hot/{symbol}` — the cockpit, max density. 70%+ of session time.
- **Agent Observatory** — `/agents/observatory` — the analyst's workbench. 15–25% of session.
- **Risk Control** — `/risk` — highest-stakes surface; must stay responsive when others degrade.
- **Backtesting** — `/backtests` + detail / compare — validate profile changes before they go live.
- **Settings** — `/settings/{section}` — CALM mode; configure intent, not react to markets.

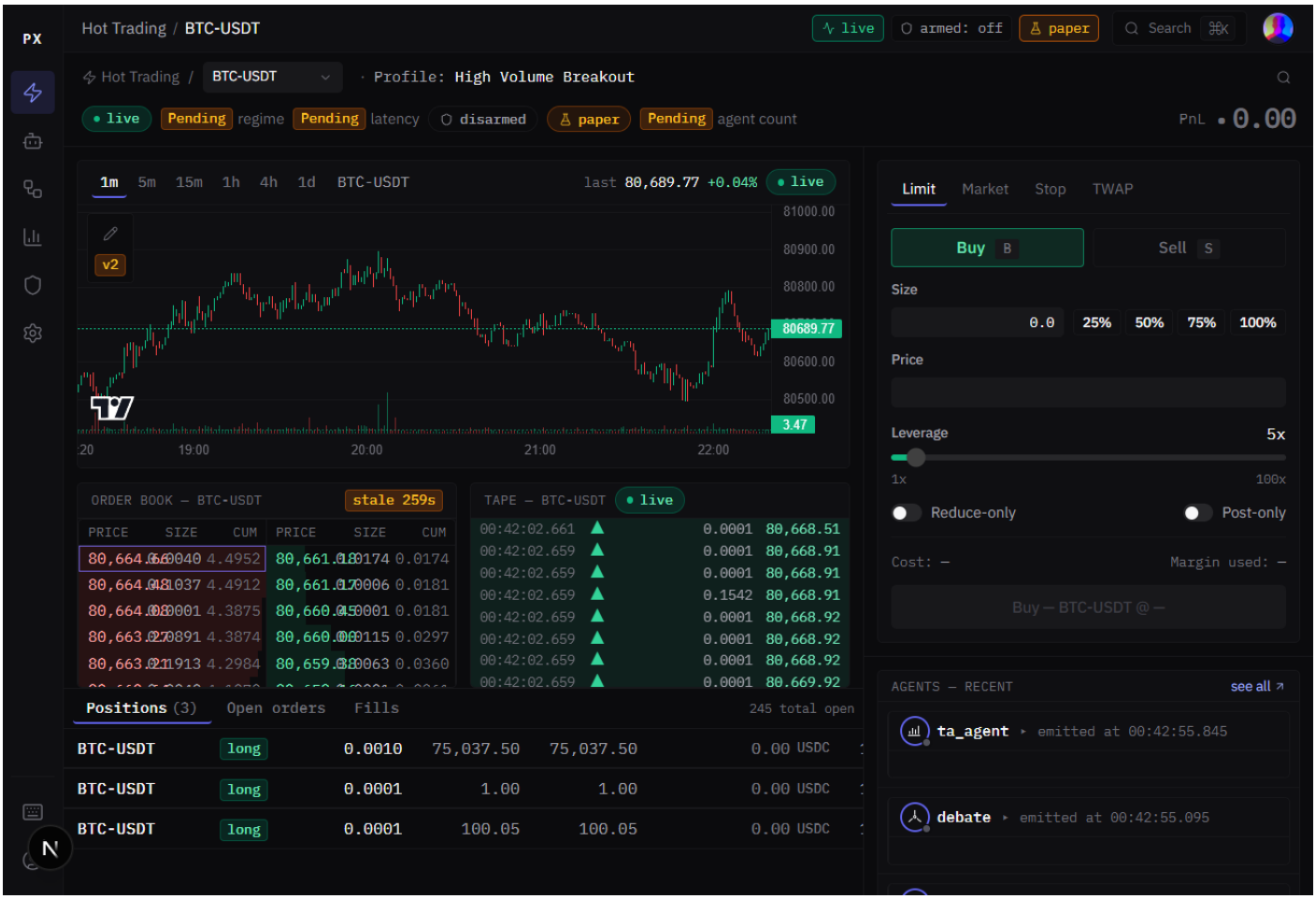
Mode contract. Every surface declares `data-mode="hot|cool|calm"` on its root. The three modes share a token vocabulary but differ in density, visual budget, and tone. HOT is the cockpit; COOL is the analyst's workbench; CALM is the office where you configure intent.

Contents

14. Settings — Sessions / API (newly wired)
15. Settings — Audit log

1. Hot Trading — cockpit

Screenshot: `hot_trading_default.png`



`/hot/{symbol}` is the surface that opens on sign-in. Three-column grid: collapsible left rail + center column (chart on top, order book + trades tape below, positions / open orders / fills tabs at the bottom) + right column (order entry on top, agent summary below). HOT mode, max density — this is the cockpit.

What the screenshot is showing

- **Chart** — lightweight-charts candle chart with timeframe selector (1m / 5m / 15m / 1h / 4h / 1d). Fluid layout so candles paint correctly across viewport sizes.
- **Order book** — 50 price levels per side, 1.0-tick aggregation default, mid + spread anchored center. DOM virtualized; never more than ~60 visible rows.
- **Trades tape** — TapeRow stream, max 200 buffered, newest at top. Auto-scroll lock if the user scrolls up.
- **Positions** tab — every open position for the active profile, refreshing on every tick. Negative unrealized in ask, positive in bid.
- **Order entry** — side toggle (B / S keys), market / limit type (M / L), size, leverage, post-only / reduce-only flags. Submits to `/orders`.
- **Agent summary** (right column, below order entry) — recent AgentTrace cards (TA, regime, sentiment); embedded DebatePanel surfaces when a debate is in flight.

- **Chrome connection pill** top-right is `/ready`-aware (ADR-017): green LIVE / amber DEGRADED / red 503 STALE.

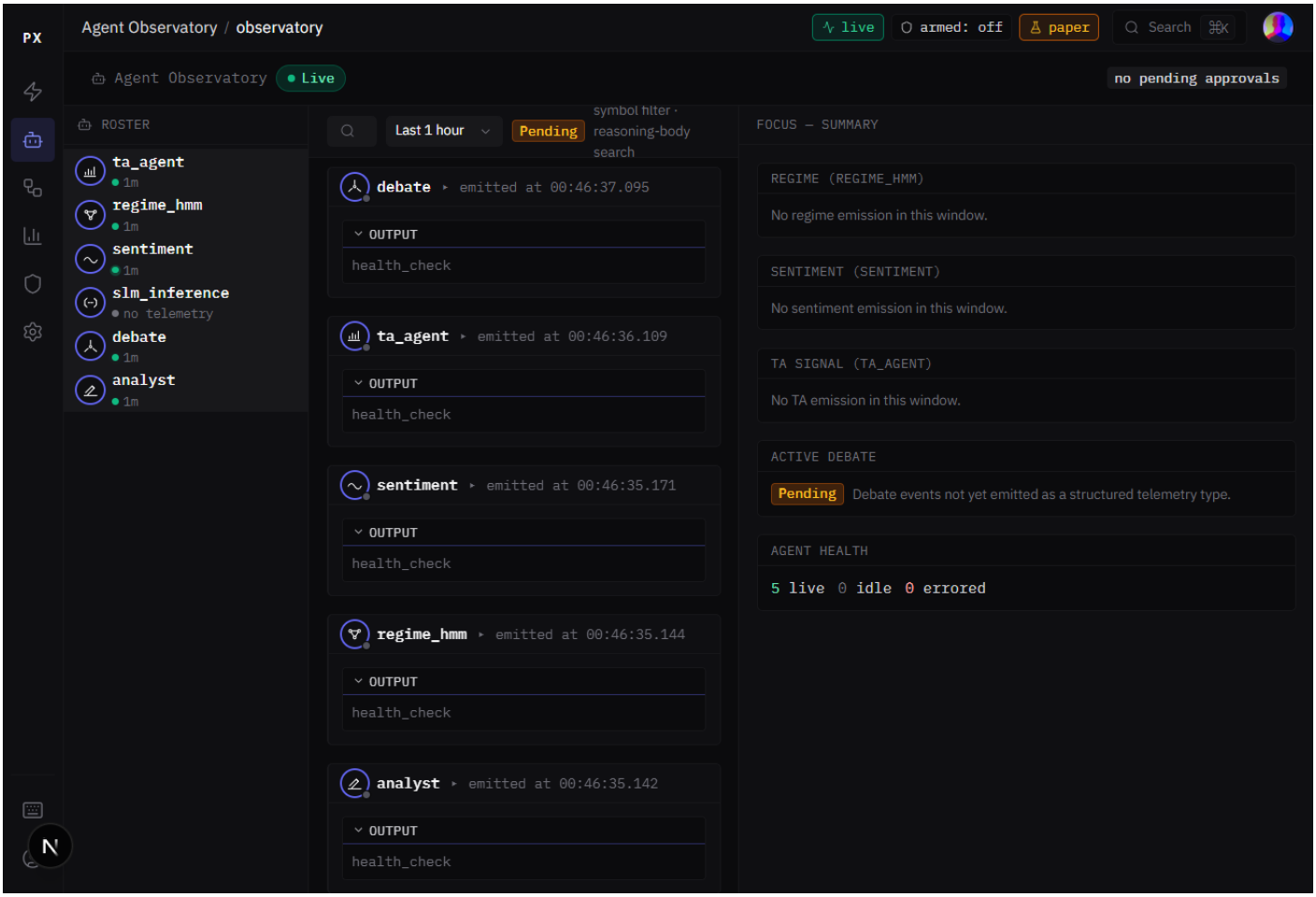
Source of truth (code)

```
frontend/app/hot/[symbol]/page.tsx   // surface composition

frontend/components/trading/PriceChart.tsx   // fluid chart + timeScale().fitContent() + monotonic-skip in update path

frontend/components/trading/OrderBook.tsx   // virtualized DOM

frontend/components/trading/TapeRow.tsx   // streaming tape

frontend/components/trading/PositionsPanel.tsx

frontend/components/trading/OrderEntryPanel.tsx

frontend/components/agentic/AgentSummaryPanel.tsx

docs/design/05-surface-specs/01-hot-trading.md   // surface spec
```

---

2. Agent Observatory

Screenshot: `observatory_default.png`



`/agents/observatory` is the analyst's workbench. Three columns: agent roster on the left (each row: agent name + StatusDot + last-emit time), chronological event stream in the middle (virtualized AgentTrace cards),

focus panel on the right. When no event is selected the focus panel shows a summary dashboard; click any trace to expand its reasoning chain.

What the screenshot is showing

- **Agent roster** (220px left) — ta_agent, regime_hmm, sentiment, slm_inference, debate, analyst. Click toggles include-this-agent filter; right-click for per-agent actions.
- **StatusDot states**: live (emitted within liveness window), idle (>5m since last emit), error (last emit was an error), disabled (toggled off).
- **Event stream** — virtualized chronological feed. Each AgentTrace card shows agent + symbol + decision + confidence + one-line summary. Click to expand inline; click the avatar to zoom right column to that agent's focus view.
- **Filter facets** at top: agent type, symbol, time window (1h / 4h / 24h / 7d), event type (decision / debate / abstain).
- **HITL approval queue** lives here (when populated) — signals held by the HITL gate appear at the top of the stream with Approve / Reject / See full trace buttons.

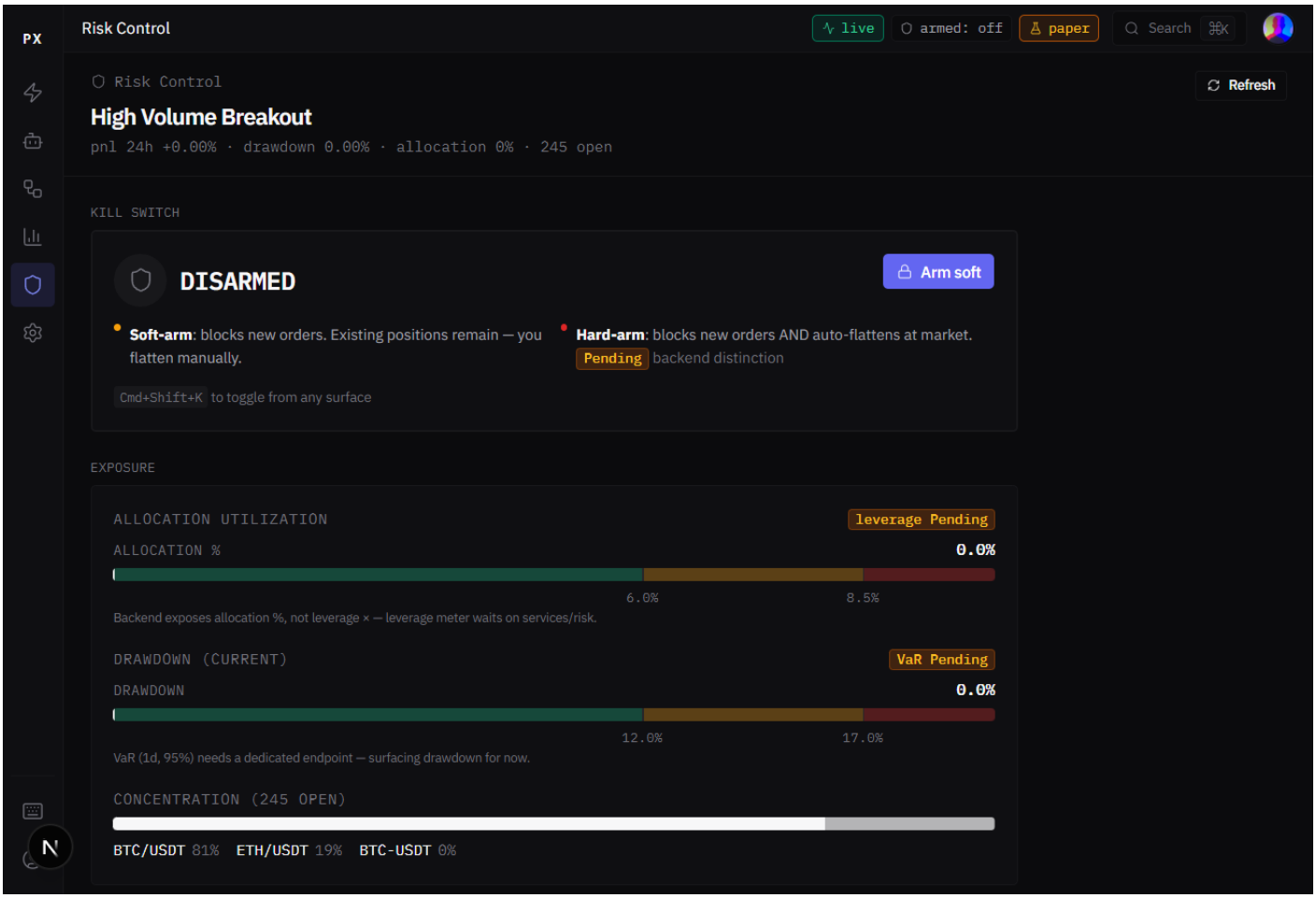
Source of truth (code)

```
frontend/app/agents/observatory/page.tsx

frontend/app/agents/observatory/_components/AgentRoster.tsx

frontend/app/agents/observatory/_components/EventStream.tsx

frontend/app/agents/observatory/_components/FocusPanel.tsx

services/api_gateway/src/routes/hitl.py   // HITL approve/reject endpoints

docs/design/05-surface-specs/02-agent-observatory.md
```

---

3. Risk Control

Screenshot: `risk_control_default.png`



`/risk` is the highest-stakes surface. Per the surface spec, this page *must remain interactive when other parts of the app are degraded* — the kill-switch keyboard shortcut (`Cmd+Shift+K` / `Ctrl+Shift+K`) and the positions read must work even if the agent system or canvas backend is unavailable.

What the screenshot is showing

- **Kill Switch card** (top) — current state (OFF / ARMED soft / ARMED hard), who set it + when + reason. Two arm actions: soft (blocks new orders, positions remain) or hard (blocks new orders AND auto-flattens at market).
- **Exposure card** — leverage as a RiskMeter, portfolio VaR (1-day 95%), concentration breakdown per symbol, current drawdown vs. peak.
- **Active limits card** — live status per limit (max position size, max leverage, daily loss limit, rate limit). Limits >60% utilized turn amber; breached turn red with a callout.
- **Recent violations** strip — last 5 rejected orders with the rule that blocked them.
- **Per-profile risk monitor** cards below (visible in the screenshot, header reads “High Volume Breakout”) sort closest-to-breach descending.

Source of truth (code)

```
frontend/app/risk/page.tsx   // surface composition

frontend/components/shell/KillSwitchModal.tsx   // confirm modal (Ctrl+Shift+K)

services/hot_path/src/kill_switch.py   // KillSwitch Redis key + reasoned arm/disarm log
```

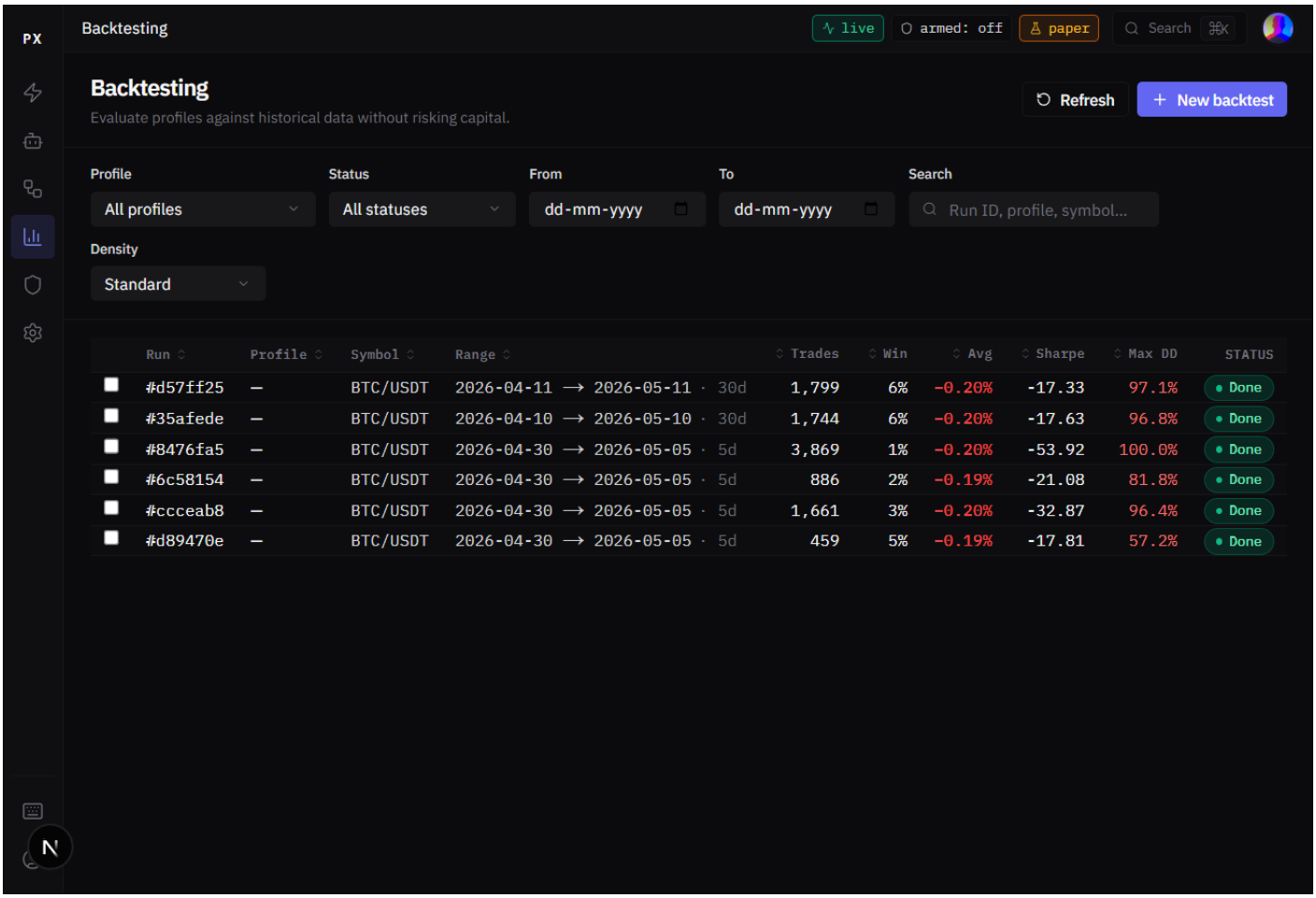
`services/api_gateway/src/routes/commands.py  // /commands/kill-switch endpoint`

`docs/design/05-surface-specs/05-risk-control.md`

---

4. Backtesting — run list

Screenshot: `backtests_list.png`



`/backtests` is the COOL-mode workbench for validating profile changes before they go live. Dense Table of every backtest run with selectable rows; selecting ≥2 rows enables the Compare action. Filter by profile, status, date range, or search by run ID.

What the screenshot is showing

- **Row columns** — run-id, profile name, symbol, date range, trades, win %, avg return, Sharpe, max DD, status.
- **Running rows** show progress and disable selection — only completed runs can be compared.
- **Selection footer** — *selected: N runs [Compare »] [Archive »] [Delete »]*. Compare is the primary action.
- **+ New backtest** (top right) opens a side drawer with the run config: symbol, date range, timeframe, slippage %, profile picker.
- Density control top-right lets the user switch between Compact / Standard / Comfortable.

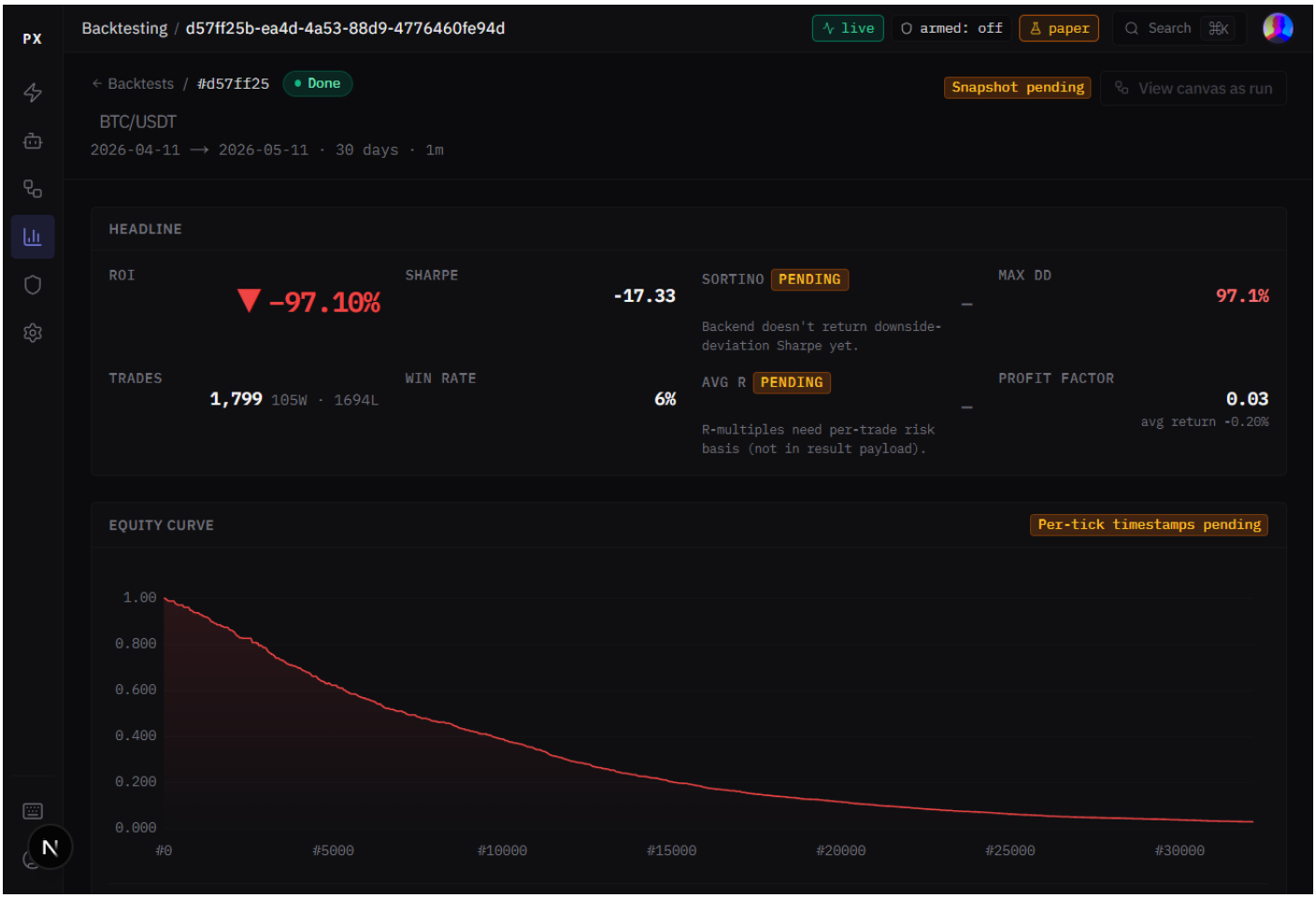
Source of truth (code)

```
frontend/app/backtests/page.tsx
frontend/app/backtests/_components/RunListTable.tsx
services/api_gateway/src/routes/backtest.py
services/backtesting/src/main.py   // worker; 2s poll, 10-min soft timeout
docs/design/05-surface-specs/04-backtesting-analytics.md
```

---

5. Backtesting — run detail

Screenshot: `backtests_detail.png`



`/backtests/{run_id}` — the post-mortem view for a single completed run. Headline metrics at the top, equity curve in the middle, per-trade table at the bottom. The screenshot shows a 30-day BTC/USDT run.

What the screenshot is showing

- **Headline** — ROI / Sharpe / Sortino / maxDD / trades / win-rate / avgR / profit-factor as StatCells.
- **Equity curve** — line chart of equity over time with drawdown overlay below.
- **Per-trade table** — every simulated trade with entry, exit, hold duration, P&L $/%, reason for close, outcome chip. Paginated.
- **Open in canvas as run** — opens the profile's Pipeline Canvas with the backtest's exact agent weights and rules pinned, so the wiring that produced this result is inspectable.

Source of truth (code)

`frontend/app/backtests/[run_id]/page.tsx`

`frontend/components/backtest/EquityCurveChart.tsx`

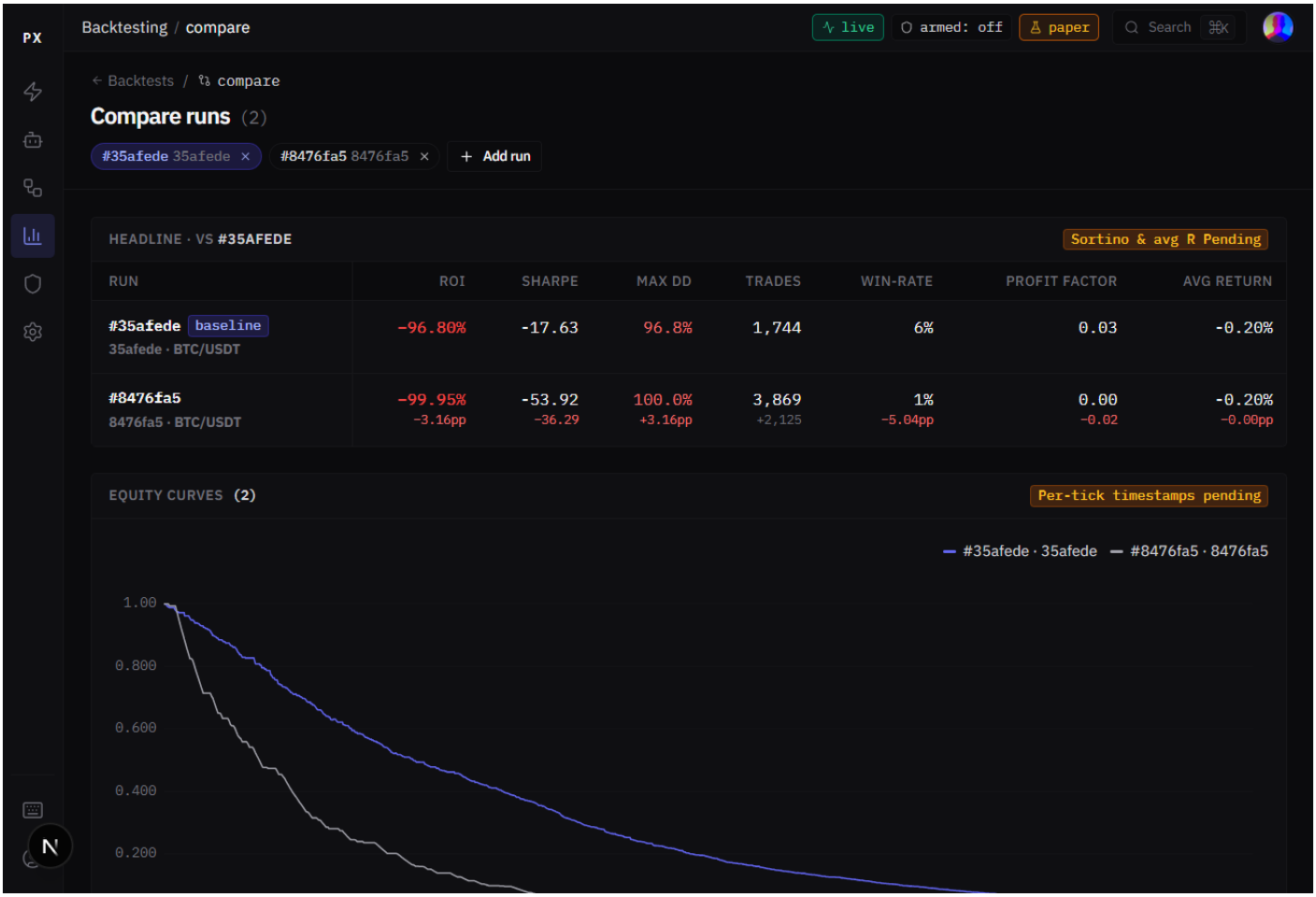
`frontend/components/backtest/TradesTable.tsx`

`libs/storage/repositories/backtest_repo.py   // migration 008→009 Decimal precision fix`

---

6. Backtesting — compare view

Screenshot: `backtests_compare.png`



`/backtests/compare?runs=A,B,C` — multiple runs overlaid on the same equity curve so divergence is visible at a glance. Below the chart: comparison table, one row per run, sortable on any metric.

What the screenshot is showing

- **Overlaid equity curves** — each run gets a distinct color; legend at the top. Synchronized cross-hair across all runs on hover.
- **Comparison table** — trades, win %, avg return, max DD, Sharpe, profit factor; cells colored relative to the best run in each column (winner highlighted).
- **Add run** button preserves the existing comparison set (`?compare=A,B,C&add=D`) — fix from commit `06dbb4a`.

- **Pin highlighted run** — clicking a row in the table makes that run the highlighted one in the simulated-trades table at the bottom.
- *Note for partner:* “Sortino & avg R Pending” tag visible on the headline panel — those metrics are emitted by the simulator next.

Source of truth (code)

`frontend/app/backtests/compare/page.tsx`

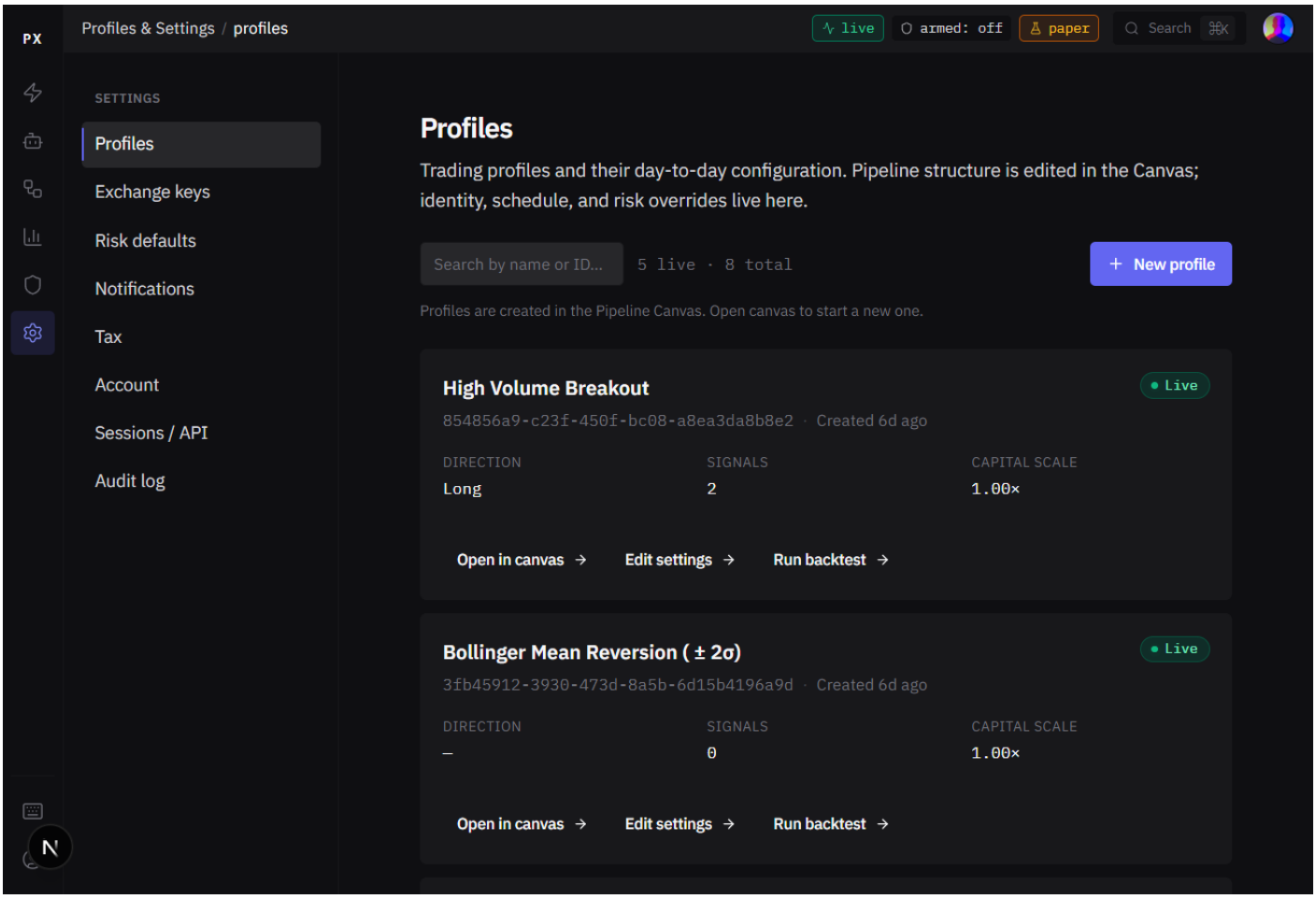
`frontend/components/backtest/ComparisonTable.tsx`

`frontend/components/backtest/EquityCurveChart.tsx  // multi-run overlay`

---

7. Settings — navigation

Screenshot: `settings_nav.png`



Settings is CALM mode — deliberately departed from HOT and COOL: generous whitespace, 15px body baseline, larger form controls, one accent color, no agent-identity colors, no live updates beyond standard save acknowledgments. The user is configuring intent here, not reacting to markets. The visual budget reflects that.

What the screenshot is showing

- **Left rail** — 8 sections: Profiles, Exchange keys, Risk defaults, Notifications, Tax, Account, Sessions / API, Audit log.

- **Content area** centered, bounded to max-width 720px. Not a full-bleed dashboard.
- **Save model** — explicit Save buttons (no auto-save, deliberately). Sticky save bar appears at the bottom when there are unsaved changes. Inline "✓ Saved" indicator for 4s after a successful save.
- **Tone** — officespace, not chatbot. No "Hi! Ready to set up?" copy.

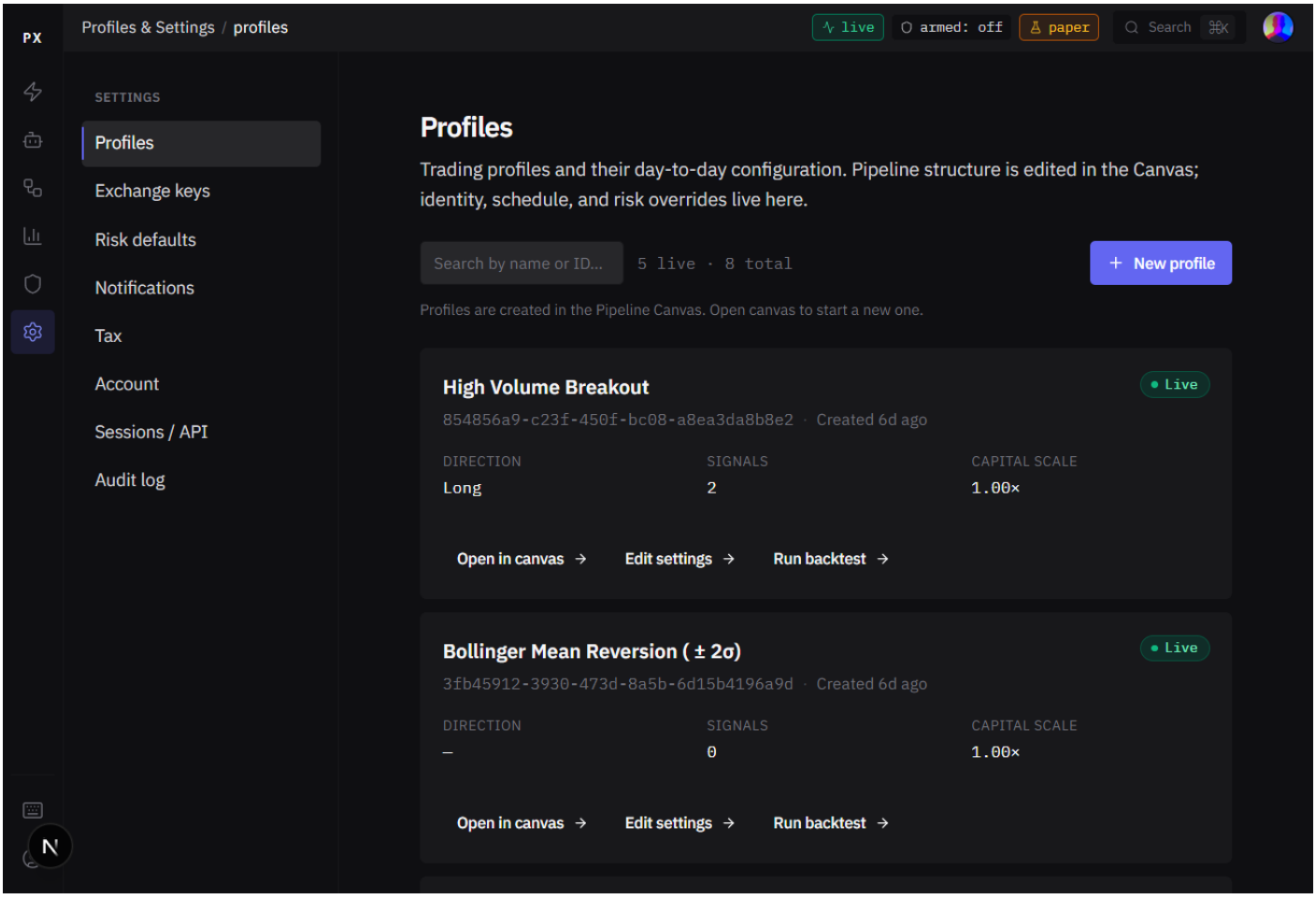
Source of truth (code)

```
frontend/app/settings/layout.tsx   // shell + section nav
frontend/app/settings/page.tsx   // landing → /settings/profiles
docs/design/05-surface-specs/06-profiles-settings.md
```

---

8. Settings — Profiles

Screenshot: `settings_profiles.png`



List of all trading profiles. Each card shows profile name, status badge (Live / Paused), last-updated, node count, 7-day P&L / trades / win-rate, and three actions: [Open in canvas] [Edit settings] [Run backtest]. Profile *structure* is edited in the Pipeline Canvas; profile *identity / overrides* is edited here. The split is by domain — behavior vs configuration.

Source of truth (code)

```
frontend/app/settings/profiles/page.tsx
```

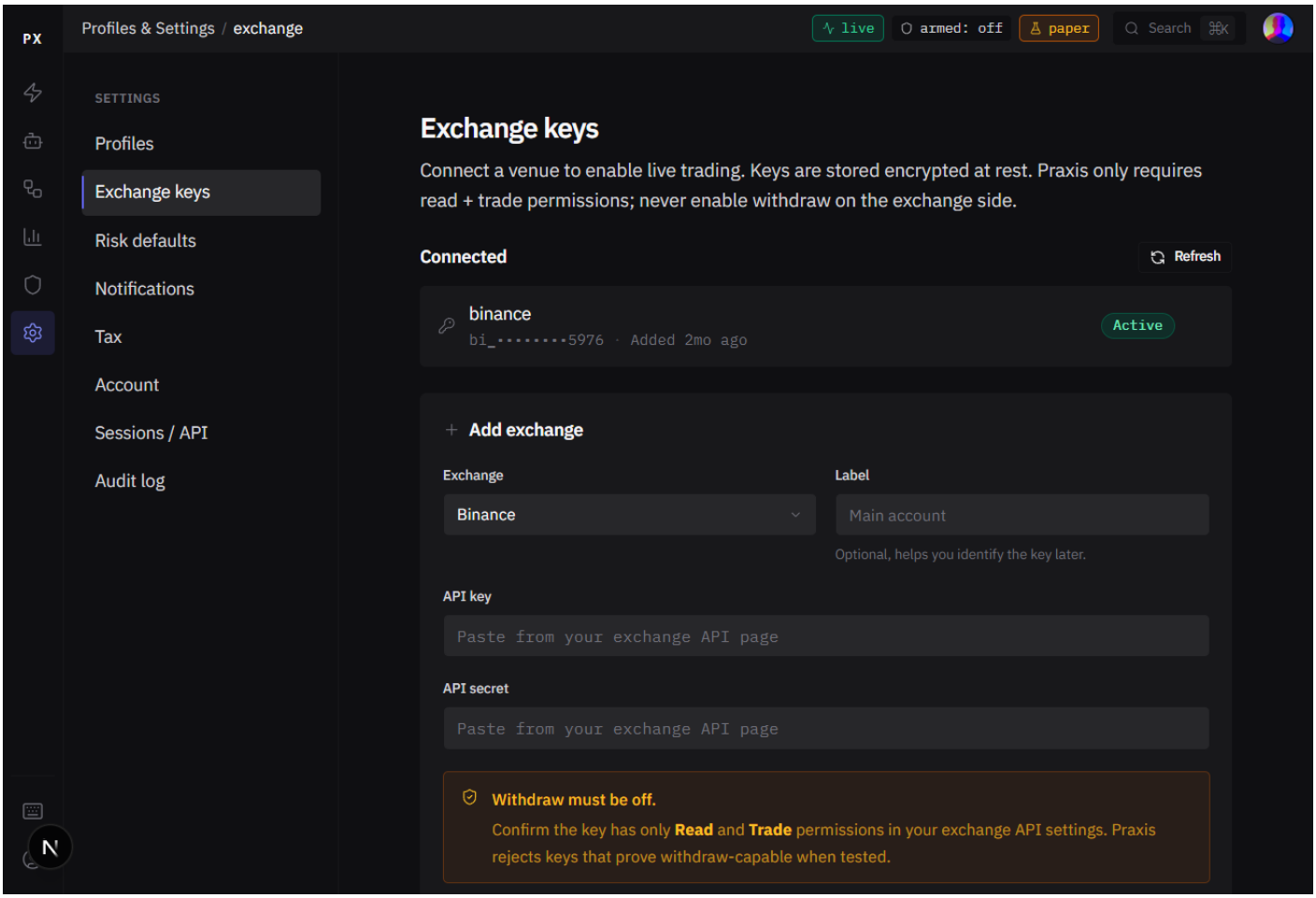
`frontend/components/settings/ProfileCard.tsx`

`services/api_gateway/src/routes/profiles.py  // CRUD endpoints`

---

9. Settings — Exchange keys

Screenshot: `settings_exchange_keys.png`



API keys for connected exchanges. Per CLAUDE.md security: Praxis never enters sensitive financial data on the user's behalf — users paste their own keys. The form makes that explicit, and the saved keys appear masked (`hl_••••••3a4f`); a saved secret is never re-displayed.

What the screenshot is showing

- **Add exchange** form — exchange dropdown, label, API key, secret, permissions checkboxes (trade / withdraw — withdraw off by default).
- **Info banner** — “Praxis never stores keys with withdraw permissions enabled by default. Confirm withdraw is off in your exchange API settings before saving.”
- **Test connection** calls CCXT `fetch_balance` to confirm the key works AND asserts withdraw is disabled (skipped on testnet, which always has full permissions).

Source of truth (code)

`frontend/app/settings/exchange/page.tsx`

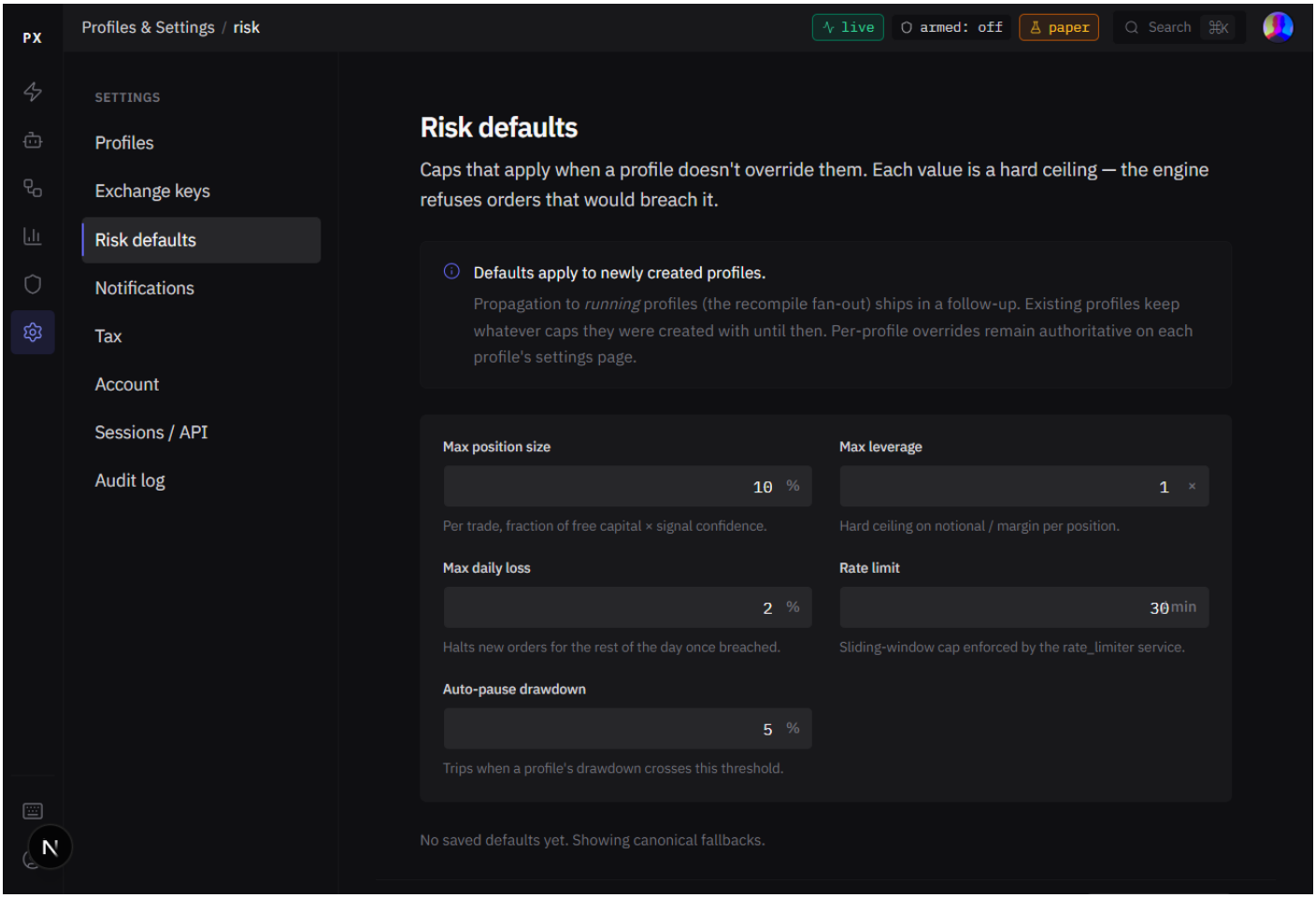
```
services/api_gateway/src/routes/exchange_keys.py   // list + create + test

libs/core/secrets.py   // GCP Secret Manager wrapper
```

---

10. Settings — Risk defaults

Newly wired (May 2026). Migration 021, repository, API route, FE form — all landed this push.

Screenshot: `settings_risk_defaults.png`



User-level risk caps that apply to *newly created* profiles. This page was previously a “shipped shell” (informational only); it was wired end-to-end in this push. The form persists; what it doesn't yet do is propagate to *running* profiles — the recompile fan-out, scoped as a separate project. The page discloses that scope inline.

What the screenshot is showing

- **Five caps**, each as a numeric input: Max position size (% of free capital × signal confidence), Max leverage (×), Max daily loss (%), Rate limit (orders / min), Auto-pause drawdown (%).
- **Inline note** at the top: “Defaults apply to newly created profiles. Propagation to *running* profiles (the recompile fan-out) ships in a follow-up.”
- **Last-saved timestamp** below the form. If never saved: “No saved defaults yet. Showing canonical fallbacks.”
- **Sticky save bar** at the bottom, visible only when dirty.

- Bounds enforced server-side via `UserRiskDefaultsPayload` (Pydantic): max_position_size_pct 0–1, max_leverage 1–20, max_daily_loss_pct 0–1, rate_limit_orders_per_min 1–600, auto_pause_drawdown_pct 0–1.

Source of truth (code)

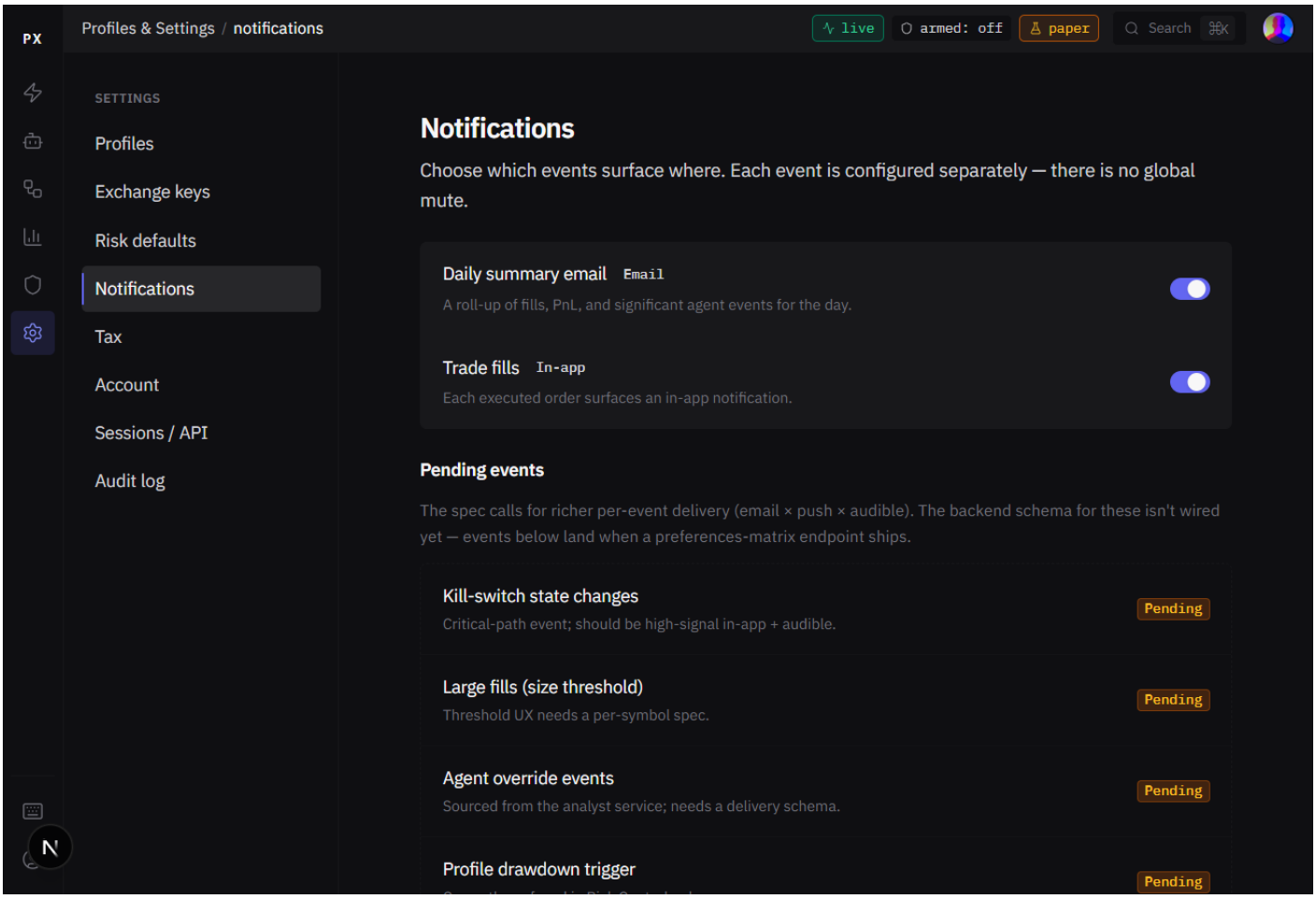
```
frontend/app/settings/risk/page.tsx   // rewritten in this push

frontend/lib/api/client.ts   // api.riskDefaults wrapper

services/api_gateway/src/routes/risk_defaults.py   // GET/PUT /risk-defaults

libs/storage/repositories/user_risk_defaults_repo.py

libs/core/schemas.py   // UserRiskDefaultsPayload + UserRiskDefaultsResponse

migrations/versions/021_user_risk_defaults.sql
```

---

11. Settings — Notifications

Partial. Two coarse booleans wired; the full event × channel matrix is the next backlog item.

Screenshot: `settings_notifications.png`



Per the spec, this is the per-event delivery matrix: each event × email / push / audible. Today the backend exposes two coarse booleans (`email_alerts`, `trade_notifications`) — those are wired. The richer matrix is Pending and the page lists each future event with a one-line reason, so the partner sees exactly

what's coming.

What the screenshot is showing

- **Active toggles** — Daily summary email, Trade fills.
- **Pending events** — Kill-switch state changes; Large fills (size threshold); Agent override events; Profile drawdown trigger; Monthly tax report ready.
- **Anti-pattern note** — no “all on/off” mega-toggle. Each event toggles separately so a stressed user can't silence everything by accident.

Source of truth (code)

`frontend/app/settings/notifications/page.tsx`

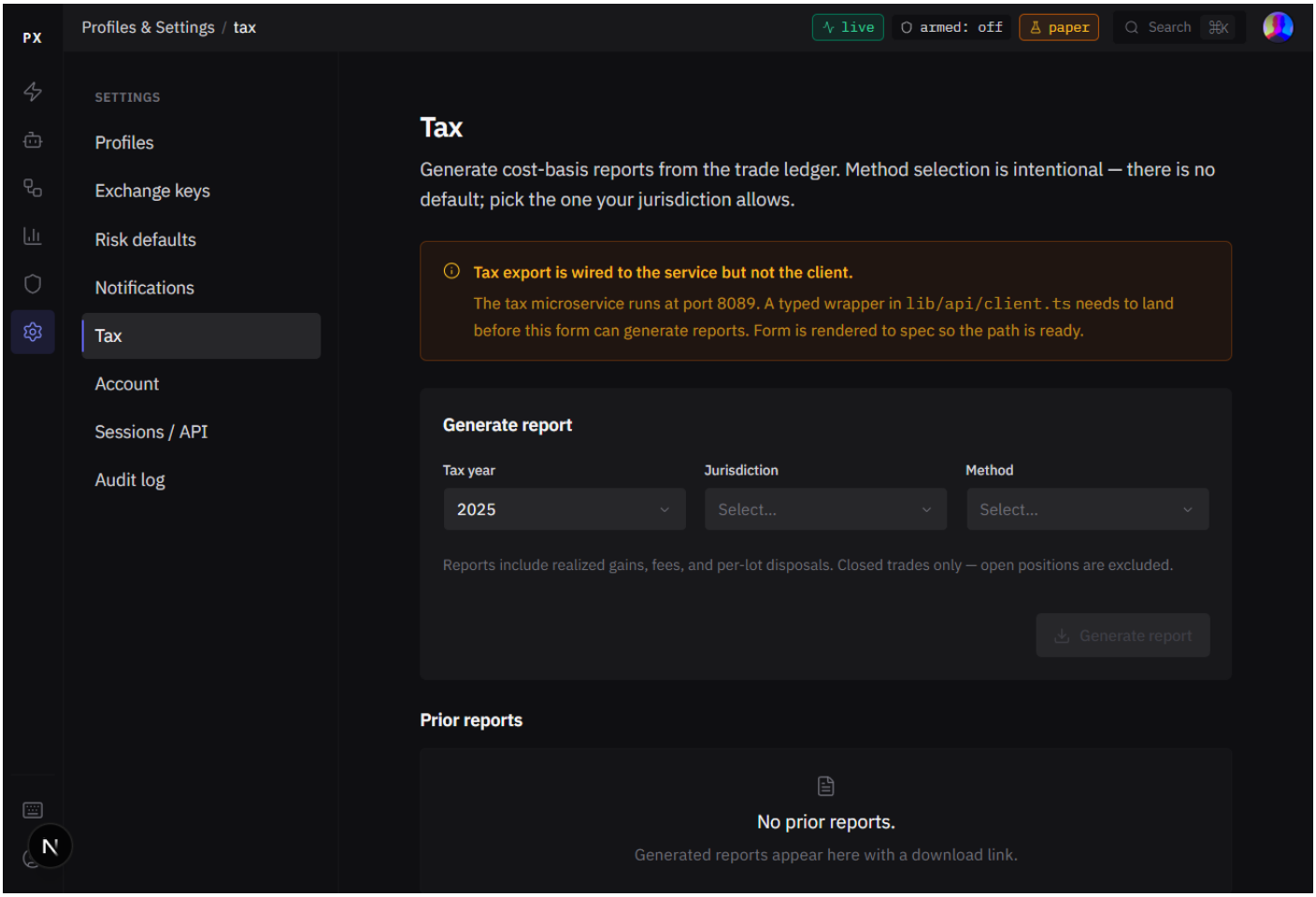
`services/api_gateway/src/routes/  // /preferences route is Pending (FE has the client wrapper; the BE route lands with the matrix schema)`

---

12. Settings — Tax

Pending. The tax microservice is a calculator today; full report generation is the next backlog item.

Screenshot: `settings_tax.png`



The form is rendered to spec. The tax microservice exists (port 8089) but currently exposes only `/calculate` and `/health` — it's a tax estimator, not a report generator. Full report generation (persistence, FIFO / HIFO /

LIFO export, year-by-year history) is its own project. The page surfaces this honestly with an inline note and a disabled Generate button.

What the screenshot is showing

- **Generate form** — year selector (last 5 years), jurisdiction (US / UK / EU / CA / AU), method (FIFO / HIFO / LIFO — intentionally no default; users must pick the one their jurisdiction allows).
- **Prior reports** section — empty state, awaits the report-generator backend.
- **Manual lot adjustments** — Pending, lands alongside the main tax client wrapper.

Source of truth (code)

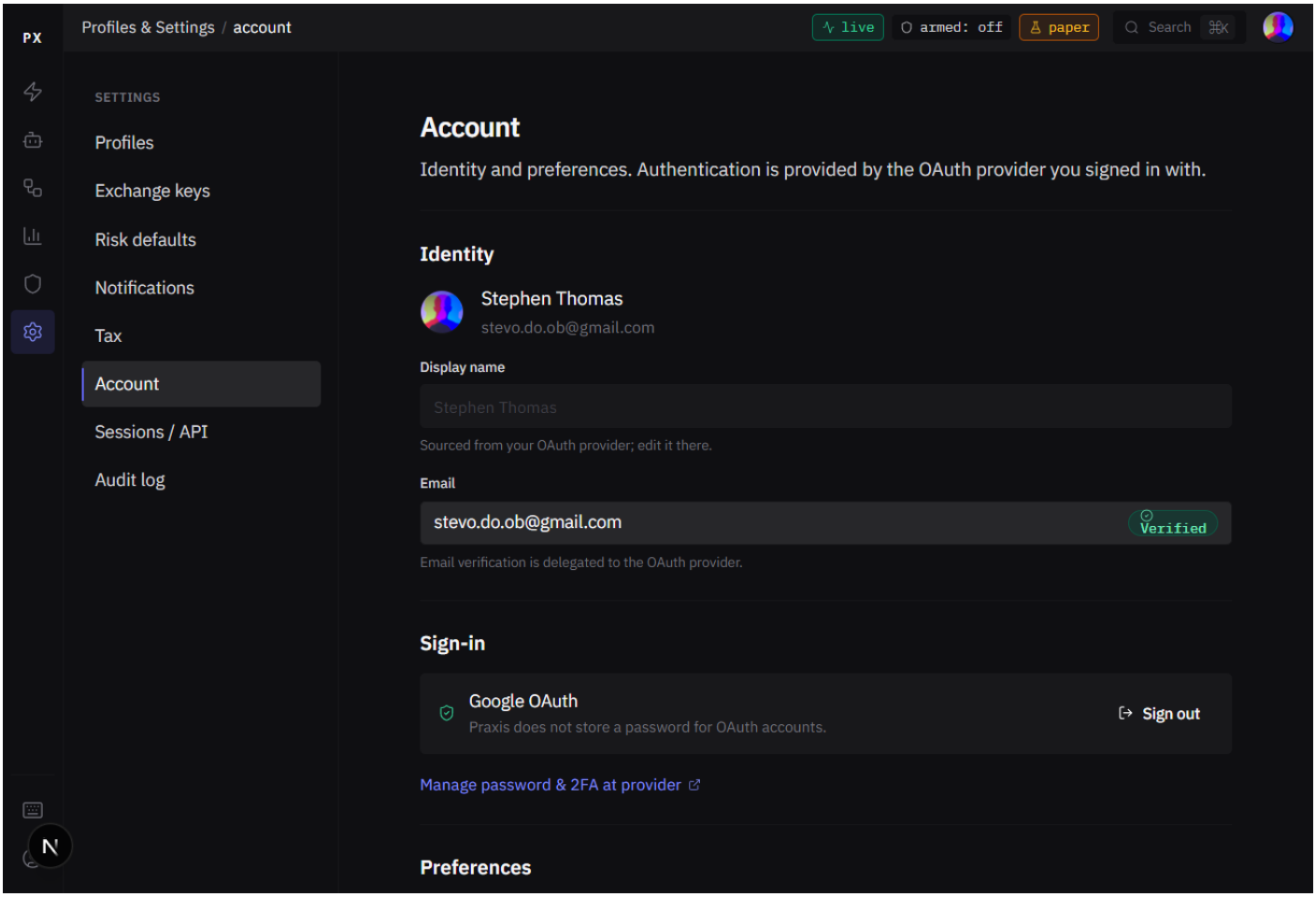
`frontend/app/settings/tax/page.tsx`

`services/tax/src/main.py  // /calculate + /health (today's surface)`

---

13. Settings — Account

Screenshot: `settings_account.png`



User identity + display preferences. Per CLAUDE.md security: never auto-fill or auto-set passwords; users type directly. Display name and avatar come from the OAuth provider (Google in this screenshot). Email shown with a verified-state pill.

Source of truth (code)

`frontend/app/settings/account/page.tsx`

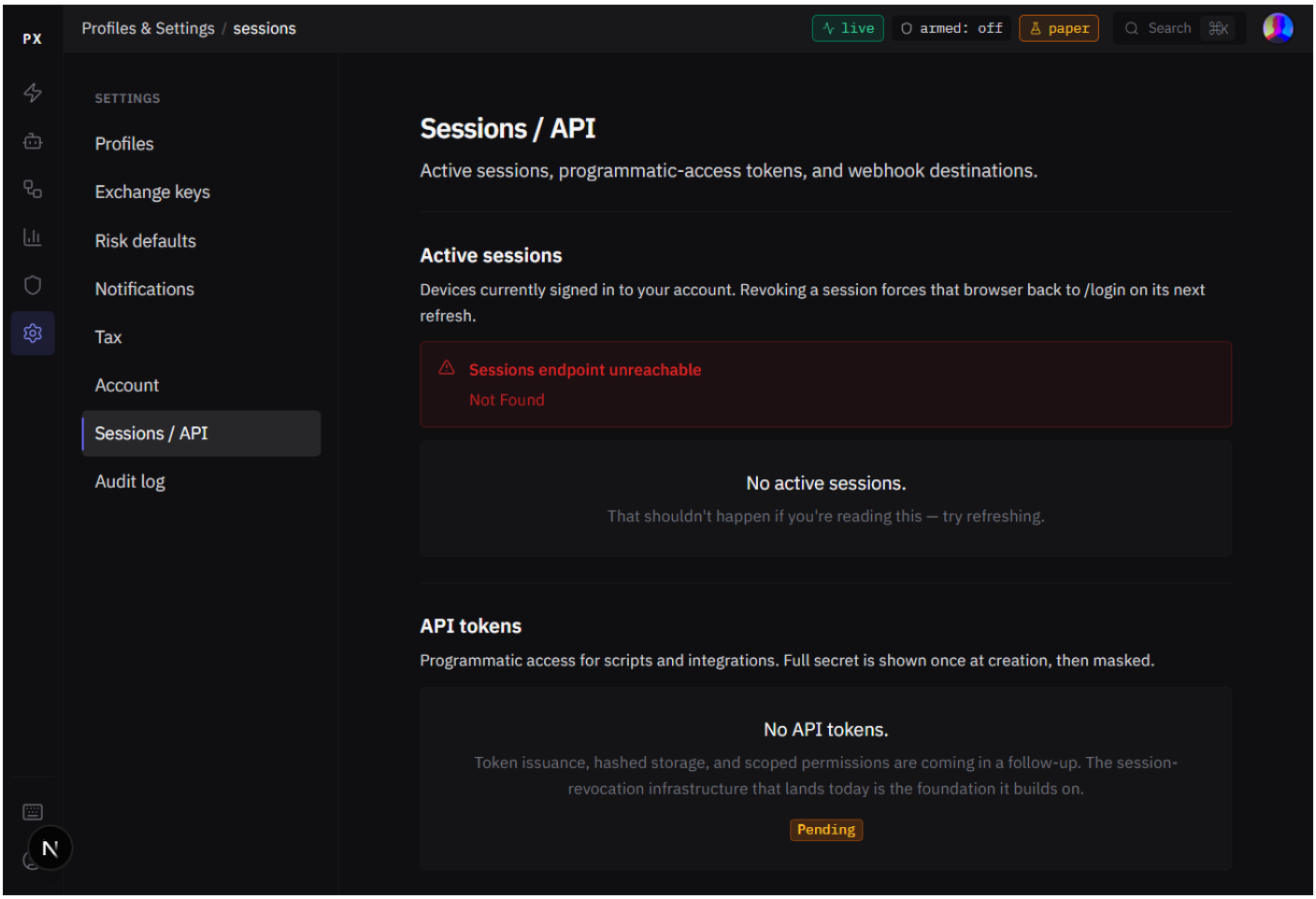
`services/api_gateway/src/routes/auth.py  // /auth/me + provider data`

---

14. Settings — Sessions / API

Newly wired (May 2026). Migration 022, repository, /auth/sessions + revoke endpoints, jti rotation on /auth/refresh, FE form — all landed this push. (Screenshot was taken before the api_gateway process was restarted to pick up the new routes — the graceful "endpoint unreachable" banner you see in the screenshot is exactly the defensive UI behavior that ships when the backend lags the FE. After restart the active sessions list populates with the user's real cross-device sessions.)

Screenshot: `settings_sessions.png`



Active sessions list. Previously single-session only ("only the current browser appears" with a Pending tag); now wired against a real `user_sessions` table populated on every `/auth/callback` and updated on every `/auth/refresh`. Any session can be revoked individually; the next refresh of that browser's token fails and the user is forced back to `/login`.

What the screenshot is showing

- **Active sessions list** — each row: device icon + browser/device label, IP + last-seen timestamp, [Sign out] on the current session and [Revoke] on others.
- **"This session" pill** — green StatusDot marks the row representing the current browser (matched via the `session_id` claim on the access token).

- **Revoke action** — flips `revoked_at` on the row; the next `/auth/refresh` for that session fails (DB is the source of truth for session liveness).
- **API tokens** section — Pending. Builds on the session-revocation pattern that landed today.
- **Webhook destinations** section — Pending, pairs with the notification matrix.

Source of truth (code)

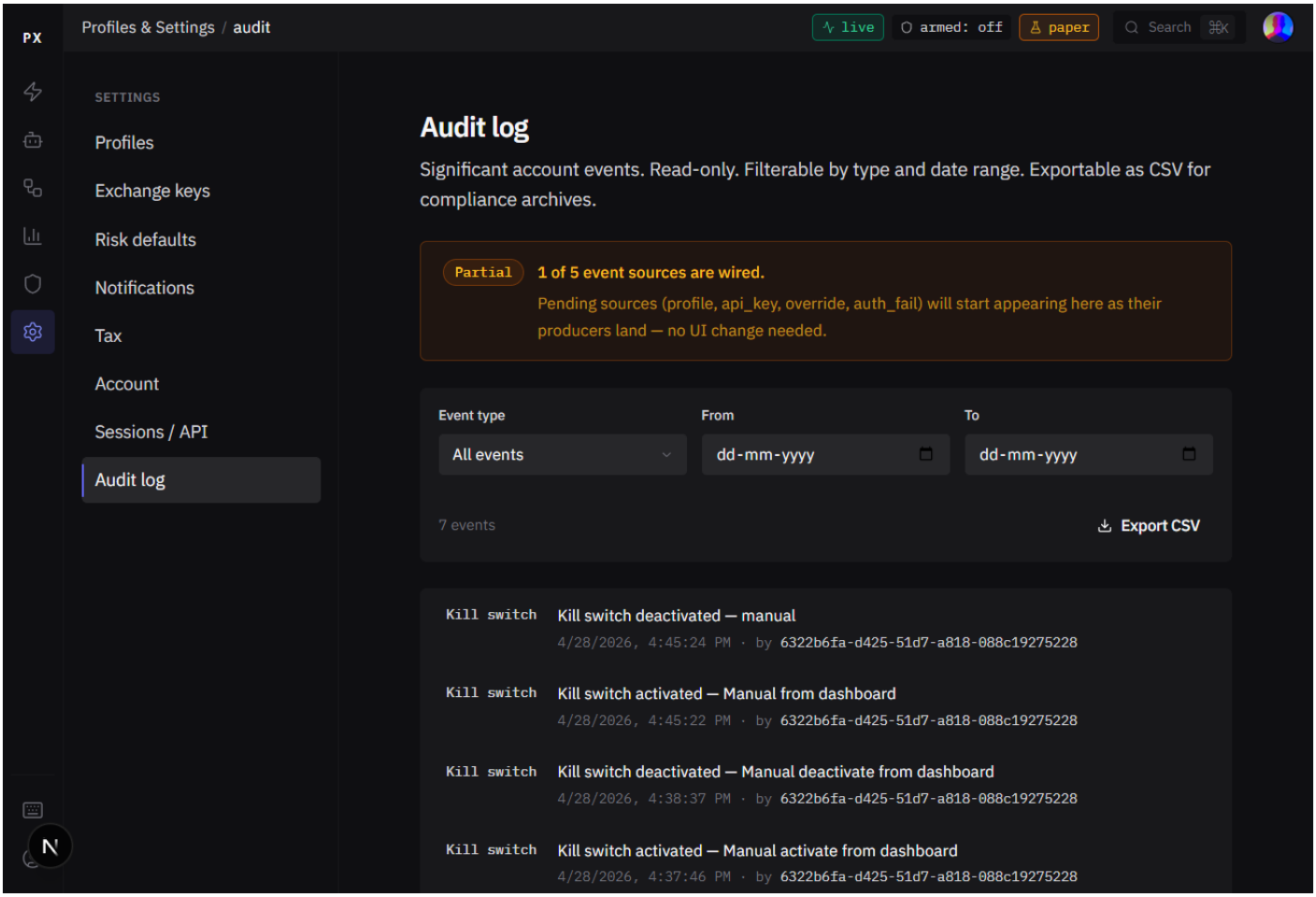
```
frontend/app/settings/sessions/page.tsx   // rewritten in this push
frontend/lib/api/client.ts   // api.sessions wrapper
services/api_gateway/src/routes/auth.py   // /auth/sessions + revoke + session create/rotate
services/api_gateway/src/middleware/auth.py   // create_refresh_token returns (token, jti)
libs/storage/repositories/user_session_repo.py
migrations/versions/022_user_sessions.sql
```

---

15. Settings — Audit log

Wired (kill-switch source). Other event sources tagged as Pending until their emitters land.

Screenshot: `settings_audit.png`



Read-only feed of significant user actions. The endpoint is wired against `services/api_gateway/src/routes/audit.py::list_user_audit_events`. Today's emitted

source: **kill-switch transitions** (from the `praxis:kill_switch:log` Redis list). Spec'd-but-not-yet-emitted sources (profile changes, API key rotations, agent overrides, failed sign-ins) stay tagged *Pending* on the page itself — the partial-feed banner tells the user how many of M sources are wired so the surface is honest about coverage.

What the screenshot is showing

- **Filter row** — event type dropdown, from-date, to-date.
- **Partial-feed banner** — “N of M event sources are wired. Pending sources will start appearing here as their producers land — no UI change needed.”
- **“What gets recorded” section** at the bottom — per-source Recorded / Pending tags so the user knows exactly which sources are emitting.
- **CSV export** at the right of the filter row.

Source of truth (code)

```
frontend/app/settings/audit/page.tsx

frontend/lib/api/client.ts   // api.audit.userEvents

services/api_gateway/src/routes/audit.py   // /user-events aggregator

services/hot_path/src/kill_switch.py   // KILL_SWITCH_LOG_KEY (today's only emitter)
```

What ships next

Five Pending items remain on the post-cutover backlog. Two of the five (Risk defaults, Sessions) were wired in this push; three remain:

- **Risk defaults — recompile fan-out** — propagate user-level saves to running profiles. Today defaults apply to new profiles only.
- **Notifications matrix** — replace the two coarse booleans with email × push × audible per event.
- **Tax report generator** — service-side persistence + FIFO / HIFO / LIFO export + year-history.
- **API tokens + webhook destinations** — on the Sessions page. Builds on the session-revocation pattern that landed today.
- **Audit log per-source emitters** — profile changes, API-key rotations, agent overrides, failed sign-ins.

Each is its own backlog project. The FE shells already render with honest Pending tags per ADR-013 — render structure, never fake. As each backend lands, the corresponding panel populates with no UI change needed.